

Spring Boot Annotations (Grouped by Category)

Spring Boot provides a large number of annotations that simplify application development. Below is a categorized list of the **most commonly used Spring Boot annotations**, along with their purpose and examples.

1. Core Spring Boot Annotations

These annotations are used to create and configure a Spring Boot application.

Annotation	Purpose
@SpringBootApplication	Main annotation that combines @Configuration, @EnableAutoConfiguration, and @ComponentScan.
@EnableAutoConfiguration	Automatically configures Spring Boot based on dependencies.
@ComponentScan	Scans packages for Spring components.
@Configuration	Declares a Java configuration class.
@Bean	Creates and manages a Spring Bean manually.

Example

```
@SpringBootApplication
public class EmployeeApplication {

    public static void main(String[] args) {
        SpringApplication.run(EmployeeApplication.class, args);
    }

}
```

2. Stereotype Annotations

These annotations tell Spring which classes should be managed as Beans.

Annotation	Used For
@Component	Generic Spring Bean
@Service	Business Logic Layer

Annotation	Used For
@Repository	Database Layer (DAO)
@Controller	Spring MVC Controller
@RestController	REST API Controller

MVC Example

```

@RestController
public class EmployeeController {

}

@Service
public class EmployeeService {

}

@Repository
public class EmployeeRepository {

}

```

3. Dependency Injection Annotations

These annotations inject dependencies automatically.

Annotation	Purpose
@Autowired	Automatically injects Bean
@Qualifier	Selects a specific Bean
@Primary	Makes a Bean the default choice
@Value	Injects values from application.properties

Example

```

@Autowired
private EmployeeService employeeService;
@Value("${server.port}")
private int port;

```

4. REST Controller Annotations

Used to create RESTful Web Services.

Annotation	Purpose
@RestController	REST Controller
@RequestMapping	Maps URL
@GetMapping	HTTP GET
@PostMapping	HTTP POST
@PutMapping	HTTP PUT
@DeleteMapping	HTTP DELETE
@PatchMapping	HTTP PATCH

Example

```
@GetMapping("/employees")
public List<Employee> getEmployees() {

}
```

5. Request Parameter Annotations

Used to receive data from client.

Annotation	Purpose
@RequestBody	Reads JSON request body
@RequestParam	Reads Query Parameter
@PathVariable	Reads URL Path Variable
@RequestHeader	Reads HTTP Header
@CookieValue	Reads Cookie
@ModelAttribute	Binds Form Data

Example

```
@GetMapping("/employee/{id}")
public Employee getEmployee(@PathVariable int id){

}
```

6. Response Annotations

Annotation	Purpose
@ResponseBody	Returns JSON
@ResponseStatus	Sets HTTP Status
ResponseEntity	Complete HTTP Response

Example

```
return ResponseEntity.ok(employee);
```

7. JPA/Hibernate Entity Annotations

These annotations map Java classes to database tables.

Annotation	Purpose
@Entity	Marks Entity Class
@Table	Maps Table Name
@Id	Primary Key
@GeneratedValue	Auto Increment ID
@Column	Maps Column
@Transient	Ignore Field
@Lob	Large Object
@Enumerated	Enum Storage
@Version	Optimistic Locking

Example

```
@Entity
@Table(name="employees")
public class Employee{

    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private int id;

}
```

8. Relationship Mapping Annotations

Annotation	Relationship
@OneToOne	One to One
@OneToMany	One to Many
@ManyToOne	Many to One
@ManyToMany	Many to Many
@JoinColumn	Foreign Key
@JoinTable	Junction Table

Example

```
@OneToMany(mappedBy="department")
private List<Employee> employees;
```

9. Validation Annotations

Used with Spring Validation.

Annotation	Validation
@Valid	Enable Validation
@NotNull	Cannot be Null
@NotBlank	Cannot be Blank
@NotEmpty	Cannot be Empty
@Size	Length Validation
@Min	Minimum Value
@Max	Maximum Value
@Positive	Positive Number
@Negative	Negative Number
@Email	Valid Email
@Pattern	Regex Validation
@Past	Past Date
@Future	Future Date

Example

```
public class Employee{

    @NotBlank
    private String name;
```

```
@Email
private String email;

}
```

10. Transaction Annotations

Annotation	Purpose
@Transactional	Database Transaction

Example

```
@Transactional
public void saveEmployee() {

}
```

11. Exception Handling Annotations

Annotation	Purpose
@ExceptionHandler	Handles Specific Exception
@ControllerAdvice	Global Exception Handler
@RestControllerAdvice	REST Exception Handler

Example

```
@RestControllerAdvice
public class GlobalException{

}
```

12. Spring Security Annotations

Annotation	Purpose
@EnableWebSecurity	Enable Security
@PreAuthorize	Method-Level Authorization
@PostAuthorize	Post Authorization
@Secured	Role-Based Access
@RolesAllowed	JSR-250 Role Check

Example

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteEmployee() {

}
```

13. Scheduling Annotations

Annotation	Purpose
@EnableScheduling	Enable Scheduler
@Scheduled	Run Scheduled Tasks

Example

```
@Scheduled(cron="0 0 9 * * ?")
public void sendEmails() {

}
```

14. Caching Annotations

Annotation	Purpose
@EnableCaching	Enable Cache
@Cacheable	Read from Cache
@CachePut	Update Cache
@CacheEvict	Remove Cache

15. Asynchronous Processing

Annotation	Purpose
@EnableAsync	Enable Async
@Async	Execute Method Asynchronously

16. Configuration Properties

Annotation	Purpose
@ConfigurationProperties	Bind Properties
@PropertySource	Load Property File

17. Testing Annotations

Annotation	Purpose
@SpringBootTest	Integration Testing
@WebMvcTest	Controller Testing
@DataJpaTest	Repository Testing
@MockBean	Mock Spring Bean
@Test	JUnit Test
@BeforeEach	Before Every Test
@AfterEach	After Every Test

18. Lombok Annotations (Commonly Used with Spring Boot)

Annotation	Purpose
@Getter	Generate Getters
@Setter	Generate Setters
@Data	Getter + Setter + Equals + hashCode + ToString
@NoArgsConstructor	No-Argument Constructor
@AllArgsConstructor	All-Argument Constructor

Annotation	Purpose
@RequiredArgsConstructor	Constructor for Required Fields
@Builder	Builder Pattern
@ToString	Generate toString()
@EqualsAndHashCode	Generate equals() and hashCode()
@Slf4j	Logger Support

19. Profiles and Environment

Annotation	Purpose
@Profile	Activate Beans for Specific Environments
@ConditionalOnProperty	Create Bean Based on Property
@ConditionalOnMissingBean	Create Bean Only if One Doesn't Exist

Quick Revision Table (Most Frequently Asked in Interviews)

Category	Important Annotations
Boot	@SpringBootApplication, @Configuration, @Bean
MVC	@Controller, @RestController, @RequestMapping, @GetMapping, @PostMapping
Dependency Injection	@Autowired, @Qualifier, @Primary, @Value
JPA	@Entity, @Table, @Id, @GeneratedValue, @Column
Relationships	@OneToOne, @OneToMany, @ManyToOne, @ManyToMany, @JoinColumn
Validation	@Valid, @NotNull, @NotBlank, @Email, @Size
Transactions	@Transactional
Exception Handling	@ControllerAdvice, @RestControllerAdvice, @ExceptionHandler
Security	@EnableWebSecurity, @PreAuthorize, @Secured
Scheduling	@EnableScheduling, @Scheduled
Caching	@EnableCaching, @Cacheable, @CachePut, @CacheEvict
Async	@EnableAsync, @Async
Testing	@SpringBootTest, @WebMvcTest, @DataJpaTest, @MockBean

Category	Important Annotations
Lombok	@Data, @Getter, @Setter, @Builder, @Slf4j

These grouped annotations cover the vast majority of annotations you'll encounter in Spring Boot development, especially when building REST APIs with MVC, Spring Data JPA, Spring Security, validation, testing, and enterprise applications.